

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа #5

Выполнил:

Косарев Илья

Группа

J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Домашняя работа 5: изучение основ работы с менеджером зависимостей npm

В рамках данной работы Вам предстоит изучить основные команды пакетного менеджера NPM и научиться стартовать проект на Vue. Научиться работать с npm и vue на основе мануала: <https://docs.google.com/document/d/187UkgGNrcWqkb2aCGpkHTLgeozoElMqdVgVGMBOC9gk/edit?usp=sharing>.

В качестве отчёта ожидаю скриншоты ваших компонентов и код получившегося приложения.

Ход работы

Сначала развернул базовый проект-пример от самого Vue (папку assets удалил, так как стили берутся из Bootstrap):

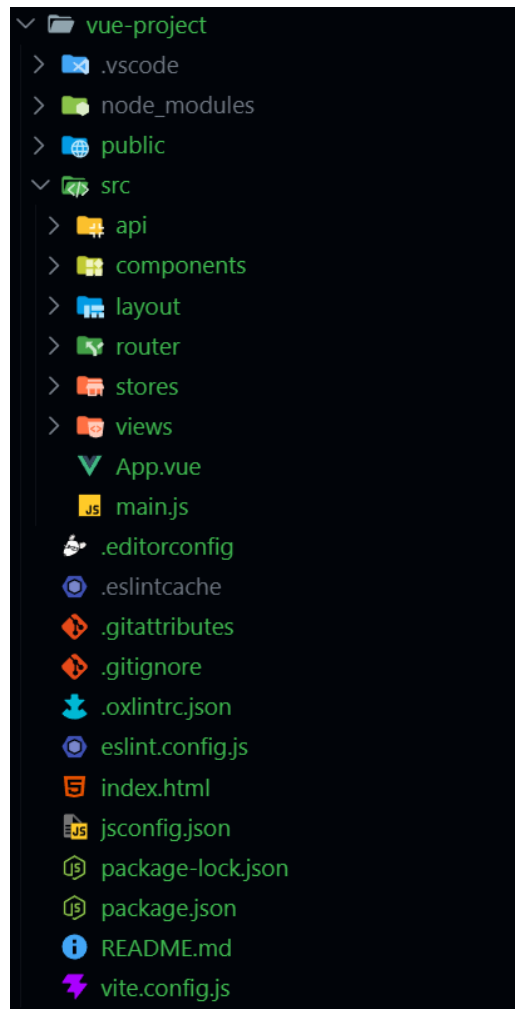


Рисунок 1: структура проекта

Настроил axios для взаимодействия с локальным json-server (установлен как зависимость отдельно, порт 3000), причем в instance.js создал единый инстанс axios для всего приложения, чтобы легче отлавливать глобальные состояния:

```
import axios from 'axios'

const apiURL = 'http://localhost:3000'

const instance = axios.create({
  baseURL: apiURL
})

export default instance
```

После этого создал notes.js для работы с API самого приложения заметок:

```
class NotesApi {
  constructor(instance) {
    this.API = instance
  }

  getAll = async () => {
    return this.API({
      url: '/notes'
    })
  }

  createNote = async (data) => {
    return this.API({
      method: 'POST',
      url: '/notes',
      data,
      headers: {
        'Content-Type': 'application/json'
      }
    })
  }
}

export default NotesApi
```

Объединим оба api в единый IoC-контейнер index.js:

```
import instance from "@/api/instance"
import NotesApi from "@/api/notes"

const notesApi = new NotesApi(instance)

export {
  notesApi
}
```

После этого настроил хранилище Pinia для управления управления глобальным состоянием:

```
import { persist } from 'pinia-persists'
import { createPinia } from 'pinia'

const pinia = createPinia()

pinia.use(persist())

export default pinia
```

После этого создал хранилище для работы с API заметок в notes.js (/stores):

```
import { defineStore } from 'pinia'
import { notesApi } from '@api'

const useNotesStore = defineStore('notes', {
  state: () => ({
    notes: []
  }),
  actions: {
    async loadNotes() {
      const response = await notesApi.getAll()
      this.notes = response.data
      return response
    },
    async createNote(data) {
      const response = await notesApi.createNote(data)
      this.notes = response.data
      return response
    }
  }
})

export default useNotesStore
```

После этого подвязал все элементы инфраструктуры в main.js.

После этого создал компонентную структуру: базовый layout BaseLayout.vue, представление главной страницы приложения заметок NotesPage.vue и компонент карточки заметки NoteCard.vue.

```

<template>
  <base-layout>
    <h1>Notes app</h1>
    <form ref="noteForm" @submit.prevent="createCard" class="d-flex flex-column my-5">
      <input type="text" v-model="form.name" class="my-1">
      <textarea cols="30" rows="10" v-model="form.text" class="my-1"/>
      <button type="submit" class="btn btn-primary">
        Отправить
      </button>
    </form>

    <div class="row row-cols-1 row-cols-md-2 g-4 mt-5" id="notes">
      <div class="col" v-for="note in notes" :key="note.id">
        <note-card :name="note.name" :text="note.text" />
      </div>
    </div>
  </base-layout>
</template>

<script>
import BaseLayout from '@/layout/BaseLayout.vue'
import NoteCard from '@/components/NoteCard.vue'
import { mapActions, mapState } from 'pinia'
import useNotesStore from '@/stores/notes'

export default {
  name: 'NotesPage',
  components: { BaseLayout, NoteCard },
  data() {
    return {
      form: {
        name: '',
        text: '',
        userId: 1
      }
    }
  },
  computed: {
    ...mapState(useNotesStore, ['notes'])
  },
  methods: {
    ...mapActions(useNotesStore, ['loadNotes', 'createNote']),

    async createCard() {
      await this.createNote(this.form)
      await this.loadNotes()

      this.$refs.noteForm.reset()
    }
  },
  mounted() {
    this.loadNotes()
  }
}
</script>

```

Рисунок 2: NotesPage.vue

```

<template>
  <div class="card">
    <div class="card-body">
      <h5 class="card-title">{{ name }}</h5>
      <p class="card-text">
        {{ text }}
      </p>
    </div>
  </div>
</template>

<script>
export default {
  name: 'NoteCard',
  props: {
    name: {
      type: String,
      required: true
    },
    text: {
      type: String,
      required: false
    }
  }
}
</script>

```

Рисунок 3: NoteCard.vue

```
<template>
  <main class="container my-2">
    <slot />
  </main>
</template>
```

Рисунок 4: BaseLayout.vue

В итоге получили рабочее веб-приложение на Vue с динамически обновляемым контентом (после двусторонней привязки через v-model, v-for и mounted):

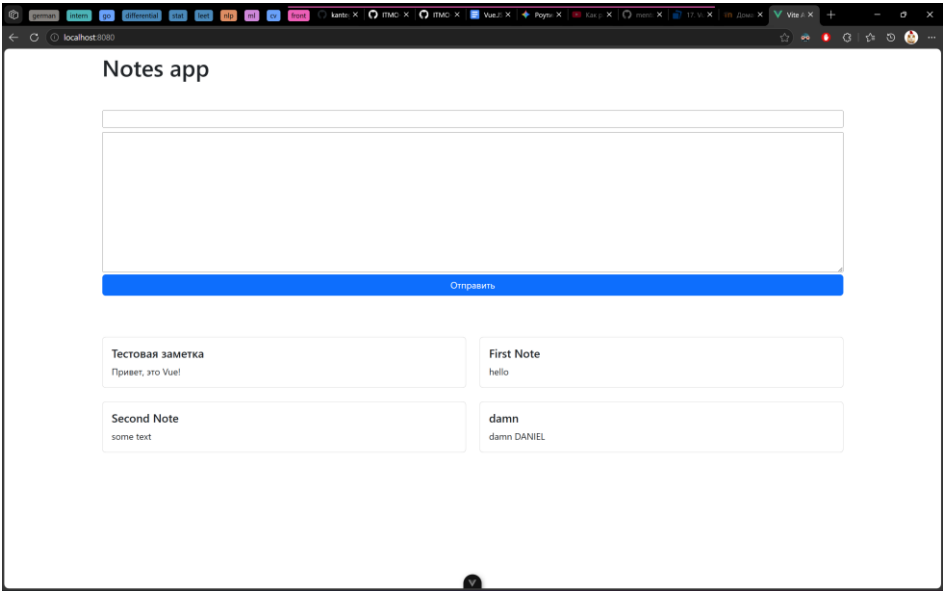


Рисунок 5: до записи заметки

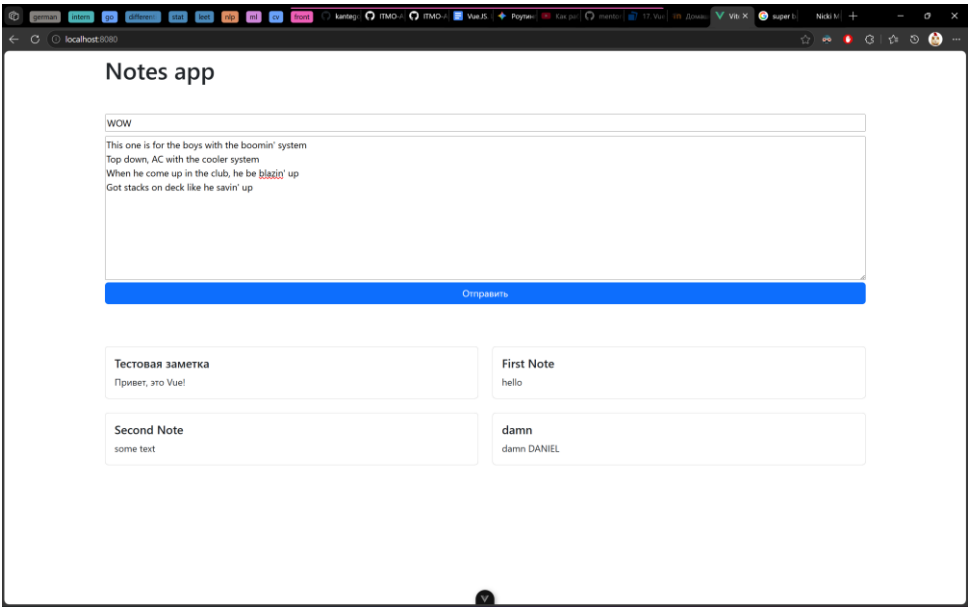


Рисунок 6: после записи в форму

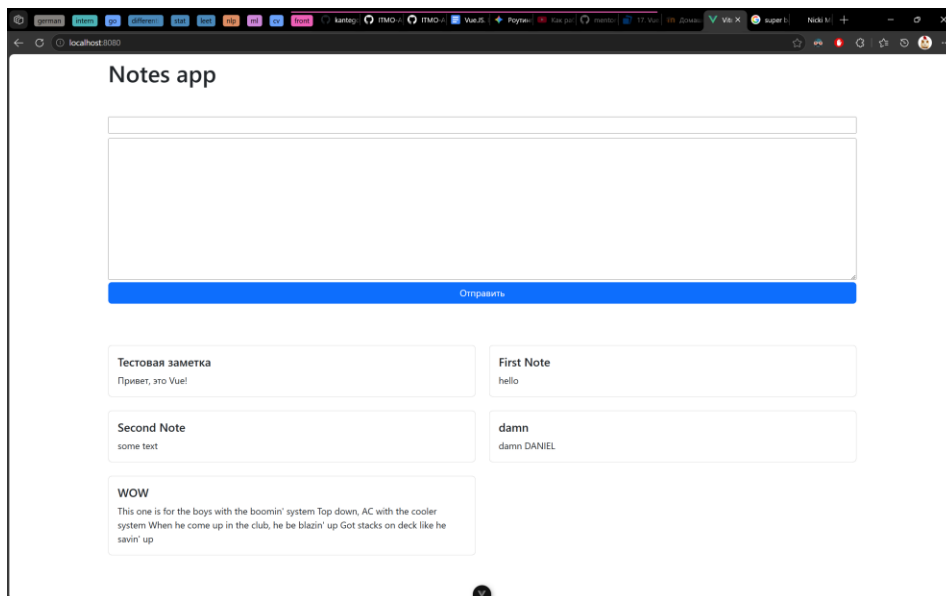


Рисунок 7: после отправки формы

Вывод

В ходе выполнения домашней работы был разработан одностраничный веб-проект на Vue.js. Были отработаны основные этапы создания приложения: инициализация проекта через NPM, настройка маршрутизации, организация компонентной структуры и реализация двустороннего связывания данных через v-model. Для управления состоянием приложения применено хранилище Pinia, что позволило централизовать работу с заметками и сделать логику интерфейса более предсказуемой.

В результате работа дала практические навыки построения SPA-приложений, понимание связки Vue + Pinia + Axios и сформировала базу для дальнейшей разработки более сложных интерфейсов и полноценной интеграции с backend-сервисами.